ResolveDesk: exact demo script and actions

Presenter account: dibyo10. Prepared on 16 October 2026. Estimated duration: 12-16 minutes, plus any fresh CI waiting time. Read the SAY paragraphs aloud; DO paragraphs are your screen actions. Timings are rehearsal targets, not timestamps in the existing video.

Before you record

DO: Open demo/slides.pdf, this guide, your editor, the terminal, http://localhost:3000 and https://github.com/dibyo10/resolvedesk-devsecops. Open demo/README.md as the evidence index. Use your normal Chrome profile for any account login. Finish login before recording. Keep credentials and notification popups out of frame.

DO: Start Docker Desktop. In the project directory run the following commands. Wait for all three services to become healthy. On another computer, first clone the repository and follow README.md for prerequisites and .env setup.

```
cd /Users/dibyochakraborty/devsecops-python
docker compose up -d --wait
docker compose ps
curl --fail http://localhost:8000/ready
python3 scripts/smoke-api.py --base-url http://localhost:8000
```

DO: Use screen recording with your microphone, and rehearse once. Present all ten slides first, then show the project. This script supplements the existing demo/walkthrough.mp4; it does not require replacing that complete recording.

IMPORTANT: AWS was destroyed after the verified cloud demonstration. Cloud URLs and kubectl commands from that session no longer work. For this rehearsal, show the genuine recorded cloud evidence and identify it as recorded. If your evaluator requires a new continuous live cloud session, a fresh deployment and teardown are required; do not describe screenshots as live. The existing walkthrough includes the actual push and CI run, cloud application and monitoring footage.

00:00-03:30 | Present all ten slides

Slide 1: ResolveDesk

DO: Show slide 1 full screen.

SAY: "This is ResolveDesk, a helpdesk application for recording, assigning and resolving support tickets. The goal is to demonstrate a small working product and its complete DevSecOps delivery process. I will cover the architecture first, then show the application, code, pipeline and recorded cloud deployment. The temporary AWS resources have already been removed."

Slide 2: A small workflow, implemented end to end

DO: Show slide 2.

SAY: "Each ticket keeps its requester, subject, description, priority and assignee together. A ticket moves from Open to In progress to Resolved, and it can be reopened. Search, filters and summary counts make triage easier. This is a classroom demonstration using synthetic data. It does not include authentication or send customer email."

Slide 3: Separate responsibilities

DO: Show slide 3.

SAY: "The React frontend handles the workspace and forms. FastAPI routes define the HTTP interface, Pydantic schemas validate requests, and service functions perform ticket operations. SQLAlchemy manages

storage, while Alembic versions the database schema. This separation makes the application easier to understand and test."

Slide 4: One application, two delivery paths

DO: Trace the diagram from browser to PostgreSQL, then point to the delivery and monitoring paths.

SAY: "A browser reaches React through Nginx. API requests go to FastAPI, which stores tickets in PostgreSQL. Docker Compose runs these services locally. In the cloud, Terraform creates the infrastructure and Helm describes the workloads. GitHub Actions publishes verified images, ArgoCD reconciles the release, and Prometheus supplies metrics to Grafana."

Slide 5: Test before publication

DO: Show slide 5.

SAY: "The pipeline runs backend and frontend tests and builds the frontend. It builds each container image once and tags it with the source commit SHA. Trivy blocks High and Critical vulnerability findings. The same scanned images are loaded into a temporary Kubernetes cluster for a Helm smoke test, then published to the GitHub Container Registry."

Slide 6: Make infrastructure reproducible

DO: Show slide 6.

SAY: "Terraform defines a VPC across two availability zones and an EKS cluster. The first NAT-based attempt hit the account's Elastic IP quota. The temporary deployment therefore used public-subnet workers with no public inbound node rules and an operator-restricted Kubernetes API. The apps ran successfully. The recovery apply was later interrupted while AWS deleted an obsolete failed worker group; that log is not claimed as a completed successful apply."

Slide 7: Deploy and recover predictably

DO: Show slide 7.

SAY: "The deployment used two frontend and two backend replicas, health probes and resource limits. PostgreSQL used a persistent gp3 volume. A migration job prepared the schema, and application readiness checked the database. The backend HPA had valid CPU measurements. These results are documented with the captured runtime output."

Slide 8: Git records the desired release

DO: Show slide 8.

SAY: "ArgoCD reads the Helm configuration from Git and reconciles the cluster with that desired state. Commit-based image tags connect the deployed image to the tested source. The captured application reached Synced and Healthy. The configuration also leaves backend replica ownership to the HPA, so ArgoCD does not continually undo scaling decisions."

Slide 9: Observe behavior, not just containers

DO: Show slide 9.

SAY: "The health endpoint checks the process, readiness checks the migrated database, and metrics expose request counts and timing. Prometheus discovered both backend replicas as Up. Grafana displayed actual request rate and latency. The server-error panel had no matching series; I do not present that empty panel as a measured zero."

Slide 10: Evidence and lessons

DO: Show slide 10.

SAY: "ResolveDesk passed twenty-six backend tests and four frontend tests. The cloud application, GitOps status and monitoring were verified before teardown. Terraform destroy removed fifty-nine resources and all eleven cleanup checks passed. The encryption key is pending its scheduled deletion on November fifteenth. Authentication, database high availability and recovery testing remain production work."

03:30-05:00 | Live local ticket workflow

DO: Open http://localhost:3000. Click New ticket. Enter Subject: VPN access for new teammate. Requester email: demo@example.com. Description: The new teammate cannot connect to the training VPN. Priority: Urgent. Status: Open. Assigned to: Demo Support. Click Create ticket.

SAY: "I am creating a synthetic urgent helpdesk request. The form validates required fields, and the saved ticket appears in the workspace. The dashboard summarizes the current ticket state."

DO: Click Edit on that ticket, change Status to In progress, then Save changes. Type VPN into the search field. Show the filtered result. Clear search, edit the ticket again, change Status to Resolved, and save.

SAY: "I can assign and track work through its lifecycle, find it with search, and resolve it when the issue is complete. Each update goes through the API and is persisted in PostgreSQL."

DO: Refresh the browser to show the resolved ticket remains. Click Delete only on the synthetic demo ticket, then confirm the browser prompt.

SAY: "A browser refresh does not remove the data. I will now delete only the ticket created for this demonstration."

05:00-06:30 | Code, tests and containers

DO: In the editor, open backend/app/routes.py, schemas.py, services.py and models.py in that order. Show backend/alembic/versions, frontend/src/components/TicketForm.jsx, frontend/src/api.js and docker-compose.yml. Show both Dockerfiles without opening .env.

SAY: "Routes receive requests, schemas validate them, service functions implement the operations, and models describe stored data. Alembic provides explicit schema migrations rather than silently creating tables at application startup. The frontend separates API requests from reusable form and table components. Compose connects the frontend, backend and database."

DO: Run these commands from the repository root. They require the existing backend virtual environment and frontend dependencies; fresh-machine installation is in README.md.

```
backend/.venv/bin/python -m pytest backend/tests -q
npm --prefix frontend test
docker compose ps
git log -8 --oneline
```

SAY: "The tests exercise ticket operations, validation and database readiness. The local containers are healthy. The Git history records separate implementation changes, and CI repeats the automated checks on pushes."

06:30-08:30 | Real pipeline evidence

DO: Open .github/workflows/ci-cd.yml, then the recorded push run at https://github.com/dibyo10/resolvedesk-devsecops/actions/runs/38902868534. Expand the test and build-scan-publish jobs. Show Trivy artifacts and the Kind deployment and smoke-test steps. The later evidence delivery also passed at https://github.com/dibyo10/resolvedesk-devsecops/actions/runs/38906204070.

SAY: "This is the actual push-triggered run preserved in the walkthrough. It completed tests, image builds, vulnerability scans, a temporary Kubernetes deployment and registry publication. I am showing a completed recorded run, not claiming this page visit triggered a new pipeline. The image tags identify the exact source

commit."

DO: For the rubric's actual push sequence, play the corresponding CI segment from demo/walkthrough.mp4, which preserves the original waiting period. If you record a fresh push instead, first make a genuine reviewed change, such as correcting your own presentation text. Save that file, inspect the diff, and run the following sequence. Replace the example file only if your actual edit is elsewhere. Do not create an empty commit or change code only to stage a demonstration.

```
git diff -- demo/demo-script.md
git add demo/demo-script.md
git commit -m "Clarify ResolveDesk presentation narration"
git push origin main
gh run list --repo dibyo10/resolvedesk-devsecops --limit 3
```

DO: Open the newly listed run in GitHub, verify its SHA matches git rev-parse HEAD, and record it until all jobs finish. A workflow rerun is not a new push. If anything fails, investigate and report the result instead of saying it passed.

08:30-10:00 | Recorded Terraform and AWS proof

DO: Open terraform/main.tf and demo/evidence/terraform-plan.log. Show aws-vpc-console.png and aws-eks-console.png, then terraform-recovery-note.txt and the end of terraform-apply.log. These files are inside demo/evidence.

SAY: "This section shows captured evidence from the temporary AWS deployment. The VPC and EKS cluster existed during the demonstration. ResolveDesk and Aryan's CampusDesk shared that cluster, with separate namespaces, Secrets and database volumes. The first networking attempt hit a quota limit. The recovered workers ran both apps, but the recovery apply was interrupted during cleanup of the failed group. The original logs preserve that limitation."

DO: Do not run terraform apply, recreate resources or sign into a different account for this evidence-only recording. A fresh live cloud demonstration requires following terraform/README.md and repeating deployment, monitoring capture and teardown; it is not needed to read the saved proof.

10:00-11:00 | Recorded Kubernetes and ArgoCD proof

DO: Open eks-pods.png, eks-services.png, eks-storage.png, eks-hpa.png, eks-ingress-application.png and argocd-synced.png from demo/evidence. Show k8s/argocd-application.yaml and helm/resolvedesk/values.yaml in the editor. If desired, play the matching cloud segment from the existing walkthrough.

SAY: "The captured cluster had two frontend replicas and two backend replicas, with PostgreSQL on persistent storage. The ingress served the working application. The HPA had valid CPU data. This ArgoCD capture shows the application Synced and Healthy, which is stronger evidence than an Application YAML file alone. These are recorded results from before teardown."

11:00-12:00 | Recorded monitoring and repair

DO: Show prometheus-targets.png, grafana-dashboard.png and troubleshooting.png from demo/evidence. Zoom until labels and graphs are legible.

SAY: "Prometheus reported both backend targets Up, and Grafana plotted real request rate and latency generated during the session. The empty server-error panel means no matching series was observed. The troubleshooting exercise reproduced an invalid image and an incorrect Service selector in an isolated namespace, repaired them, verified HTTP access and removed the temporary lab."

12:00-13:00 | Cleanup and closing

DO: Show demo/evidence/terraform-destroy-summary.png and aws-cleanup-audit.png. Open demo/README.md so the reviewer can locate the video, slides and raw logs.

SAY: "Terraform destroy completed with fifty-nine resources removed. The live cleanup audit passed all eleven checks: the cluster and VPC were absent, all four instances were terminated, application volumes were deleted, and Terraform state was empty. The owned KMS key remains pending its scheduled deletion on November fifteenth, twenty twenty-six. No unrelated resources were changed."

SAY: "The repository contains the code, tests, infrastructure, monitoring configuration and reproducible evidence. This project used the supplied infrastructure reference and an assisted implementation process. CampusDesk shares the documented base and temporary infrastructure, while adding campus-specific behavior. The demonstration is complete; production use would require authentication, backups and a stronger database availability design. Thank you."

What to do after recording

DO: Watch the exported MP4 once. Check that all ten slides appear first, text is readable, your voice is audible, the ticket workflow succeeds, and recorded evidence is described accurately. Preserve the actual CI push footage. Keep the existing complete walkthrough unless your new video covers the required sequence successfully.

DO: Ensure README.md, the presentation PDF/PPTX, video and evidence are accessible in the public repository. Submit https://github.com/dibyo10/resolvedesk-devsecops in the form after signing into the correct student Google account. Confirm any identity fields displayed by the form; those fields have not yet been inspected. No form submission is performed by this guide.